Recuperação de Falhas

Adaptado dos
materiais da professora Raqueline Penteado

Sumário

- Introdução
- Log
- Registro adiantado em log
- Atualização no disco
  - *Steal* / *No-steal*
  - *Force* / *No-force*
- *Checkpoint*
- Técnicas de Recuperação: falhas não catastróficas
  - Atualização Adiada
  - Atualização Imediata

Introdução

- Se uma transação for executada por um SGBD, este deve apresentar as seguintes garantias:
  - **Todas as operações na transação** foram completadas com sucesso e seu efeito poderá ser **gravado permanentemente** no BD;
  - A **transação não terá efeito** sobre o BD ou sobre quaisquer outras transações.
- **Observação:** sempre pode ocorrer uma falha durante a execução de uma transação.

Introdução

Tipos de Falhas

1) Queda de sistema
   - Considerando **hardware**, **software** ou **rede**.

2) Operação na transação
   - **Divisão por zero** ou **parâmetros incorretos**.

3) Erros locais ou **condições de exceção**, que podem ser detectados pela transação.

4) Cancelamento de uma transação por **imposição do controle de concorrência**.
   - Impasse.

Introdução

Tipos de Falhas

5) Falha de disco
   - Blocos de disco podem **perder seus dados** por causa do **mau funcionamento** durante a **leitura** ou **escrita**.

6) Problemas físicos e catástrofes
   - **Falha de energia**, **fogo**, **furto**…

Introdução

Observação

- Para os **tipos de falha entre 1 e 4**, espera-se que o sistema **mantenha informações suficientes** para a **recuperação**.

Introdução

- O **gerenciador de recuperação** de um SGBD é responsável por garantir duas **propriedades importantes** das transações:

  - **Atomicidade**
    - Se necessário, **desfazer as ações** das **transações** que **não** foram **efetivadas**.

  - **Durabilidade**
    - Garantir que **todas as ações** das **transações** efetivadas **sobrevivam às falhas**.

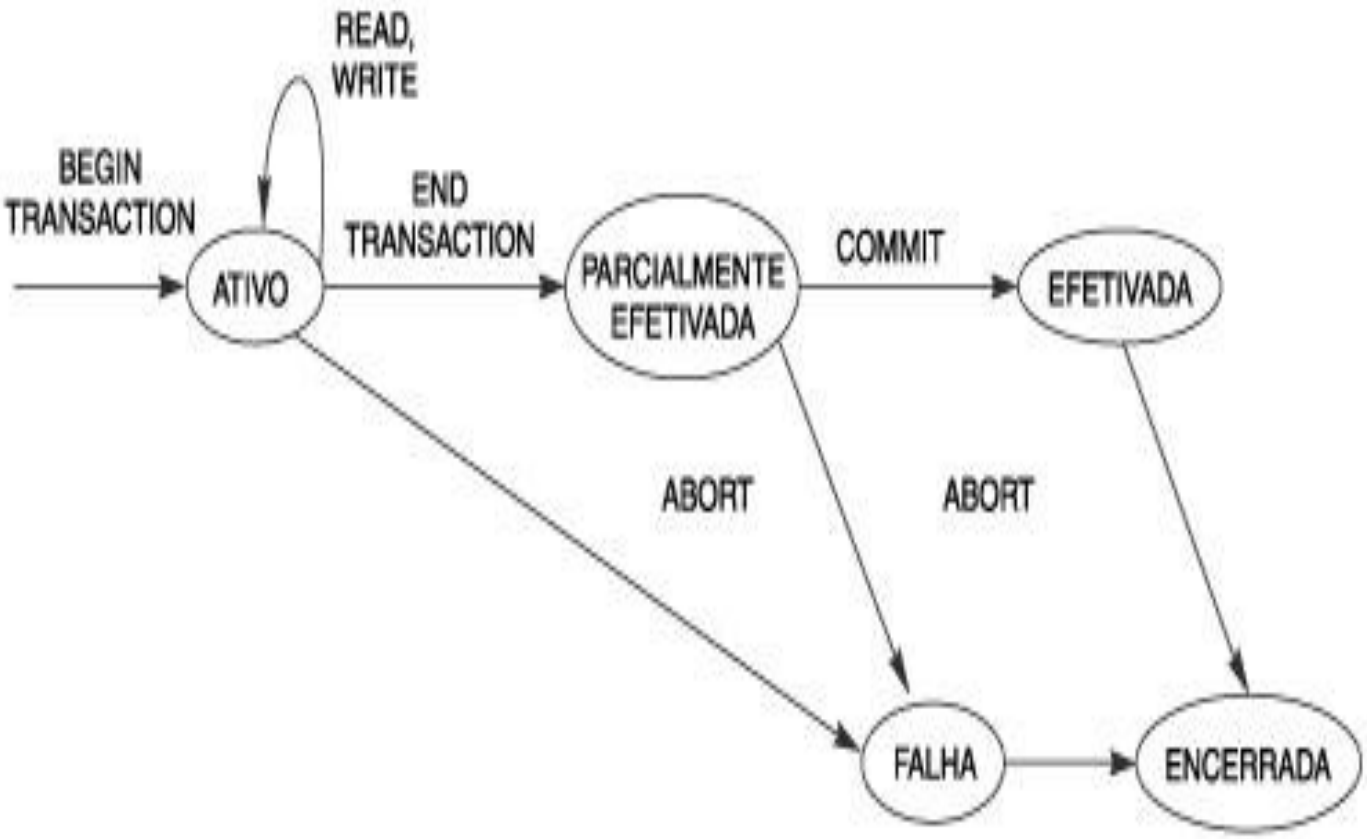
Recuperação de Falhas

- A **recuperação de falhas** de transações consiste em um processo que garante o **retorno do Banco de Dados** ao seu **estado consistente** mais recente, antes da **ocorrência de uma ou mais falhas**.

Log

- **Registro de ocorrências**.
- Arquivo especial armazenado em **memória secundária**.
- Mantém **informações** de **todas as operações**, associadas com as **transações**, que afetam os itens de dados do BD.
- **Utilizado pelos mecanismos de recuperação de falhas** para retornar o BD para um estado consistente.

Log


READ,
WRITE
BEGIN
TRANSACTION
ATIVO
END
TRANSACTION
PARCIALMENTE
EFETIVADA
COMMIT
EFETIVADA
ABORT
ABORT
FALHA
ENCERRADA

Log

- **Operações associadas com as transações**
  - ***Begin_transaction***
    - Início da execução de uma transação.
  - ***Read* ou *Write***
    - Operações de leitura ou escrita nos itens de dados do BD.
  - ***End_transaction***
    - Final da execução de uma transação.
      - Deve ser verificado se a transação executará um *commit* ou um *rollback*.
  - ***Commit_transaction***
    - Indica que a transação foi finalizada com sucesso.
    - Alterações podem se tornar permanentes no BD.
  - ***Rollback***
    - Indica que a transação foi finalizada sem sucesso.
    - Alterações nos itens de dados devem ser descartadas.

Log

- **Entradas**
  - [start_transaction, T]
  - [write_item, T, X, **old_value, new_value**]
  - [read_item, T, X]
  - [commit, T]
  - [rollback, T]

- **Sendo T**
  - **Identificador único** atribuído a cada transação do sistema.

Técnicas de Recuperação

- **Principais Técnicas para Recuperação de Falhas**
  - Atualização Adiada
    - Algoritmo NO-UNDO / REDO
  - Atualização Imediata
    - Algoritmo UNDO / REDO
    - Algoritmo UNDO / NO-REDO

Técnicas de Recuperação

- Uma **técnica de atualização adiada** considera a **atualização física** do banco de dados no **disco** somente **após** a **confirmação de uma transação**.

- Uma **técnica de atualização imediata** considera a **atualização física** do banco de dados no disco a **partir de operações** de uma transação, **antes** que a mesma **atinja seu ponto de confirmação**.

Registro Adiantado em Log

- Ao descarregar um ***buffer*** **modificado** para o **disco**, a estratégia **atualização local** geralmente é utilizada.

- Esta estratégia **grava o *buffer* de volta** para a **mesma localização no disco**, sobrescrevendo os valores antigos de quaisquer itens de dado alterados no disco.

Registro Adiantado em Log

- Nesse contexto, torna-se necessário, utilizar um *log* para recuperação.

- O **mecanismo de recuperação** deve garantir que a **imagem anterior** do **item de dado** seja **registrada** na **entrada apropriada do log** e que tal **entrada seja descarregada no disco**, antes que a **imagem anterior seja sobrescrita** pela **imagem posterior** no disco.

Registro Adiantado em Log

- Este processo é conhecido como **registro adiantado em *log***, ou ***write-ahead logging***, que consiste **gravação do *log* antes que a atualização seja efetuada** no banco de dados.

Atualização do Disco

- As **regras** ***steal/no-steal*** e ***force/no-force*** são usadas para especificar **quando** uma **página**, ou **bloco de disco**, pode ser **gravada em disco**.

- **Possibilidades**
  - ***Steal***
  - ***No-steal***

Atualização do Disco

- **Possibilidades**
  - ***Steal:*** uma **página atualizada** do buffer pode ser **gravada em disco antes do *commit*** da transação.

  - ***No-steal*:** uma **página atualizada** do buffer **não pode ser gravada em disco antes do *commit*** da transação.

Atualização do Disco

- As **regras** ***steal/no-steal*** e ***force/no-force*** são usadas para especificar **quando** uma **página**, ou **bloco de disco**, pode ser **gravada em disco**.

- **Possibilidades**
  - ***Force***
  - ***No-force***

Atualização do Disco

- **Possibilidades**
  - ***Force*:** todas as **páginas atualizadas** por uma transação **são imediatamente gravadas** em disco, quando a transação atinge seu ponto de confirmação.

  - ***No-force*:** as **páginas atualizadas** por uma transação **não são necessariamente gravadas** em disco quando a transação atinge seu ponto de confirmação.

Atualização do Disco

- Os SGBDs geralmente empregam uma estratégia **steal/no-force** no processo de recuperação de falhas.
  - Ao adotar a estratégia ***steal***, se evita a necessidade de um **espaço muito grande de *buffer*** para o armazenamento em memória de todas as páginas atualizadas.
  - A vantagem de adotar a estratégia ***no-force*** é que uma página atualizada por uma transação confirmada poderá ainda estar no *buffer* quando **outra transação considerar a atualização** da mesma, eliminando o custo de I/O para ler novamente essa página no disco.

Checkpoint

- Periodicamente, o **Sistema Operacional** força a **gravação em disco** de **todos os *buffers* do SGBD** que tenham sido alterados.
  - Quando isso ocorre, um registro chamado **ponto de verificação**, ou ***checkpoint***, é gravado em *log*.
  - O **intervalo entre *checkpoints*** pode ser medido em **unidades de tempo** ou em **número de transações confirmadas**, desde o **último *checkpoint***.

Checkpoint

- **Vantagem:** todas as transações que registraram um ***commit*** no *log* antes de um ***checkpoint*** não precisam ser refeitas em caso de falha.

Checkpoint

- **Ações na realização do *checkpoint***
  - **Suspender a execução** das transações temporariamente.
  - **Forçar a escrita** de todas as operações das transações finalizadas com sucesso, **dos *buffers* de memória principal para o disco**.
  - **Escrever a entrada** *checkpoint* no *log*.
  - **Ativar a execução** das transações suspensas temporariamente.

Reversão de Transações

- Se uma **transação T falhar** por qualquer razão, depois de **atualizar o Banco de Dados**, pode ser necessário **reverter a transação**.

- Se alguma **transação T for revertida**, qualquer **transação S** que tenha lido o valor de algum item de dado X escrito por T também deverá ser revertida.
  - **Reversão em cascata**.

Reversão de Transações

(a)

T_1
read_item(A)
read_item(D)
write_item(D)

T_2
read_item(B)
write_item(B)
read_item(D)
write_item(D)

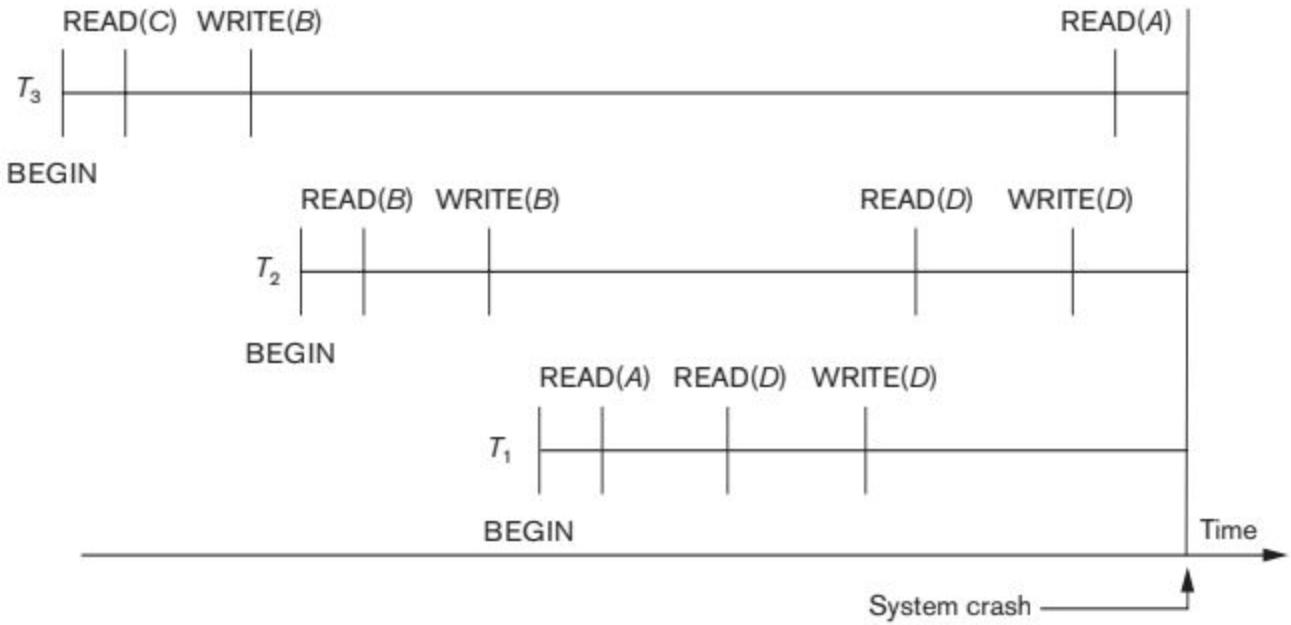
T_3
read_item(C)
write_item(B)
read_item(A)
write_item(A)

(b)

		A	B	C	D
		30	15	40	20
	[start_transaction,T_3]				
	[read_item,T_3,C]				
*	[write_item,T_3,B,15,12]		12		
	[start_transaction,T_2]				
	[read_item,T_2,B]				
**	[write_item,T_2,B,12,18]		18		
	[start_transaction,T_1]				
	[read_item,T_1,A]				
	[read_item,T_1,D]				
	[write_item,T_1,D,20,25]				25
	[read_item,T_2,D]				
**	[write_item,T_2,D,25,26]				26
	[read_item,T_3,A]				

← System crash

Reversão de Transações



Reversão de Transações

(a)

T_1	T_2	T_3
read_item(A)	read_item(B)	read_item(C)
read_item(D)	write_item(B)	write_item(B)
write_item(D)	read_item(D)	read_item(A)
	write_item(D)	write_item(A)

(b)

		A	B	C	D
		30	15	40	20
	[start_transaction,T_3]				
	[read_item,T_3,C]				
*	[write_item,T_3,B,15,12]		12		
	[start_transaction,T_2]				
	[read_item,T_2,B]				
**	[write_item,T_2,B,12,18]		18		
	[start_transaction,T_1]				
	[read_item,T_1,A]				
	[read_item,T_1,D]				
	[write_item,T_1,D,20,25]				25
	[read_item,T_2,D]				
**	[write_item,T_2,D,25,26]				26
	[read_item,T_3,A]				

← System crash

* T_3 is rolled back because it did not reach its commit point.

** T_2 is rolled back because it reads the value of item B written by T_3.

Técnicas de Recuperação

- **Atualização Adiada**
  - Adia quaisquer atualizações no BD para **depois** que a transação atingir o seu **ponto de *commit***.
  - Somente **depois deste ponto** poderá ser realizada a **atualização no BD**.

- **Atualização Imediata**
  - Permite que o BD seja atualizado por qualquer operação da transação, **antes** desta atingir o seu **ponto de *commit***.

Técnicas de Recuperação

- **Falha de Sistema**
  - ***Undo*: desfaz o efeito** das **operações *write*** de T, percorrendo o ***log* do fim para o começo** e alterando os valores do **item de dado X** para ***old_value***.

    [write_item, T, X, old_value, new_value]

  - ***Redo*: refaz o efeito** das **operações *write*** de T percorrendo o ***log* a partir do início de T até o seu fim** refazendo todos os valores de X para ***new_value***.

Técnica de Atualização Adiada

- **Antes da efetivação**
  - Alterações armazenadas no ***log*** e nos ***buffers*** **da transação**.
  - Influência da **capacidade do *buffer***.

- **Durante a efetivação**
  - Alterações primeiro armazenadas no ***log*** de maneira permanente e somente depois escritas no BD.
    - **Gravação antecipada do log**.

Técnica de Atualização Adiada

- **Se uma transação falhar antes do ponto de *commit***
  - Não é necessário desfazê-la, pois não alterou valores no BD.

- **Se ocorrer qualquer falha depois que uma transação atingiu seu ponto de *commit***
  - É necessário refazê-la.

Técnica de Atualização Adiada

- **Algoritmo Simples** (**REDO/NO-UNDO**)
  - Mantenha duas **listas de transações**
    - Uma para transações que atingiram o seu **ponto de *commit*** desde o último ***checkpoint* realizado**
      - **Lista 1**;
    - Outras para transações ativas
      - **Lista 2**;
  - Aplique a operação **REDO** a todas as **operações de escrita** das transações da lista 1 com base no ***log***.
    - Na mesma ordem em que elas foram escritas no *log*.
  - Reinicialize (**NO-UNDO**) as transações da lista 2.

Técnica de Atualização Adiada

- **Algoritmo para a operação REDO**
  - Refaça as **operações de escrita** examinando as entradas [*write_item*, T, X, *new_value*] no *log*.
  - Para cada entrada, **altere** o **valor** do **item de dado X** para ***new_value***.
  - Aqui as seguintes informações não são necessárias
    - Entrada [*read_item*, T, X]
    - *old_value* na entrada

      [*write_item*, T, X, *old_value*, *new_value*]

Técnica de Atualização Adiada

- Suponha que usemos o protocolo de atualização adiada usando o *log* abaixo, quais operações do *log* serão refeitas e quais serão ignoradas?

(a)

T_1	T_2	T_3	T_4
read_item(A)	read_item(B)	read_item(A)	read_item(B)
read_item(D)	write_item(B)	write_item(A)	write_item(B)
write_item(D)	read_item(D)	read_item(C)	read_item(A)
	write_item(D)	write_item(C)	write_item(A)

(b)

[start_transaction, T_1]
[write_item, T_1, D, 20]
[commit, T_1]
[checkpoint]
[start_transaction, T_4]
[write_item, T_4, B, 15]
[write_item, T_4, A, 20]
[commit, T_4]
[start_transaction, T_2]
[write_item, T_2, B, 12]
[start_transaction, T_3]
[write_item, T_3, A, 30]
[write_item, T_2, D, 25] ← System crash

Técnica de Atualização Adiada

(a)

T_1
read_item(A)
read_item(D)
write_item(D)

T_2
read_item(B)
write_item(B)
read_item(D)
write_item(D)

T_3
read_item(A)
write_item(A)
read_item(C)
write_item(C)

T_4
read_item(B)
write_item(B)
read_item(A)
write_item(A)

(b)

[start_transaction, T_1]
[write_item, T_1, D, 20]
[commit, T_1]
[checkpoint]
[start_transaction, T_4]
[write_item, T_4, B, 15]
[write_item, T_4, A, 20]
[commit, T_4]
[start_transaction, T_2]
[write_item, T_2, B, 12]
[start_transaction, T_3]
[write_item, T_3, A, 30]
[write_item, T_2, D, 25] ← System crash

T_2 and T_3 are ignored because they did not reach their commit points.

T_4 is redone because its commit point is after the last system checkpoint.

Técnica de Atualização Imediata

- Existe a possibilidade do **BD ser atualizado** por **qualquer operação da transação** antes desta **atingir** seu **ponto de *commit***.

- Armazenam-se as **alterações do *log***, em disco, **antes** de **aplicá-las ao BD**.

- Se uma **transação falhar depois de realizar mudanças** no BD e **antes do seu ponto de *commit***, então é **necessário desfazê-la** (UNDO).

Técnica de Atualização Imediata

- **Duas categorias de algoritmo**
  - *UNDO/NO-REDO*
    - **Estratégia *force*:** todas as **operações** de uma **transação** são **armazenadas** no BD **antes** que esta escreva a entrada [*commit*, T] no *log*.
    - No *log*, **transações com *commit* não precisam ser refeitas**, pois todas as suas alterações já foram refletidas no BD e as **transações não efetivadas precisam ser desfeitas**.
  -

Técnica de Atualização Imediata

- **Duas categorias de algoritmo**
  - UNDO/REDO
    - **Estratégia *steal/no-force*:** uma transação **pode** escrever a entrada [*commit*, T] no *log* **antes** que todas as suas alterações **tenham sido refletidas** no BD.

    - No *log*, **transações com *commit* são refeitas**, pois ainda pode haver alterações que não foram refletidas no BD e as **transações não efetivadas são desfeitas**.

Técnica de Atualização Imediata

- **Algoritmo simples (REDO/UNDO)**
  - Mantenha **duas listas de transações**
    - Uma para transações que atingiram o seu **ponto de *commit*** desde o último ***checkpoint* realizado**
      - **Lista 1**;
    - Outra para transações ativas
      - **Lista 2**;
  - Aplique a **operação REDO** a todas as **operações de escrita** nas transações da **Lista 1** com base no ***log***, seguindo a **mesma ordem de escrita**.
  - Aplique a **operação UNDO** a todas as **operações de escrita** das transações da **Lista 2** com base no ***log***, em **ordem inversa** à de **escrita no *log***.

Técnica de Atualização Imediata

- **Algoritmo para a operação REDO**
  - Refaça as **operações de escrita** examinando as entradas [*write_item*, T, X, *new_value*] no *log*.
  - Para cada entrada, **altere** o valor do **item de dado X** para ***new_value***.
  - Aqui as seguintes informações não seriam necessárias
    - Entrada [*read_item*, T, X]
    - old_value na entrada

      [*write_item*, T, X, *old_value*, *new_value*]

Técnica de Atualização Imediata

- **Algoritmo para a operação UNDO**
  - Desfaça as **operações de escrita** examinando as entradas [*Write_item*, T, X, *old_value*, *new_value*] no *log*.
  - Para cada entrada, altere o valor do **item de dado X** de ***new_value*** para ***old_value***.

Considerações Finais

- Neste material, os principais assuntos relacionados com a **recuperação de falhas** foram apresentados.

- **Estratégias de Atualização**
  - *Steal* / *No-steal*
  - *Force* / *No-force*

- **Técnicas de Recuperação**
  - Atualização Adiada
  - Atualização Imediata

Referências

- Sistemas de Banco de Dados. Elsmari e Navathe. Pearson-Addison Wesley, Sexta Edição.